université PARIS-SACLAY | IUT D'ORSAY

LMF — Laboratoire Méthodes Formelles

CentraleSupélec | université PARIS-SACLAY

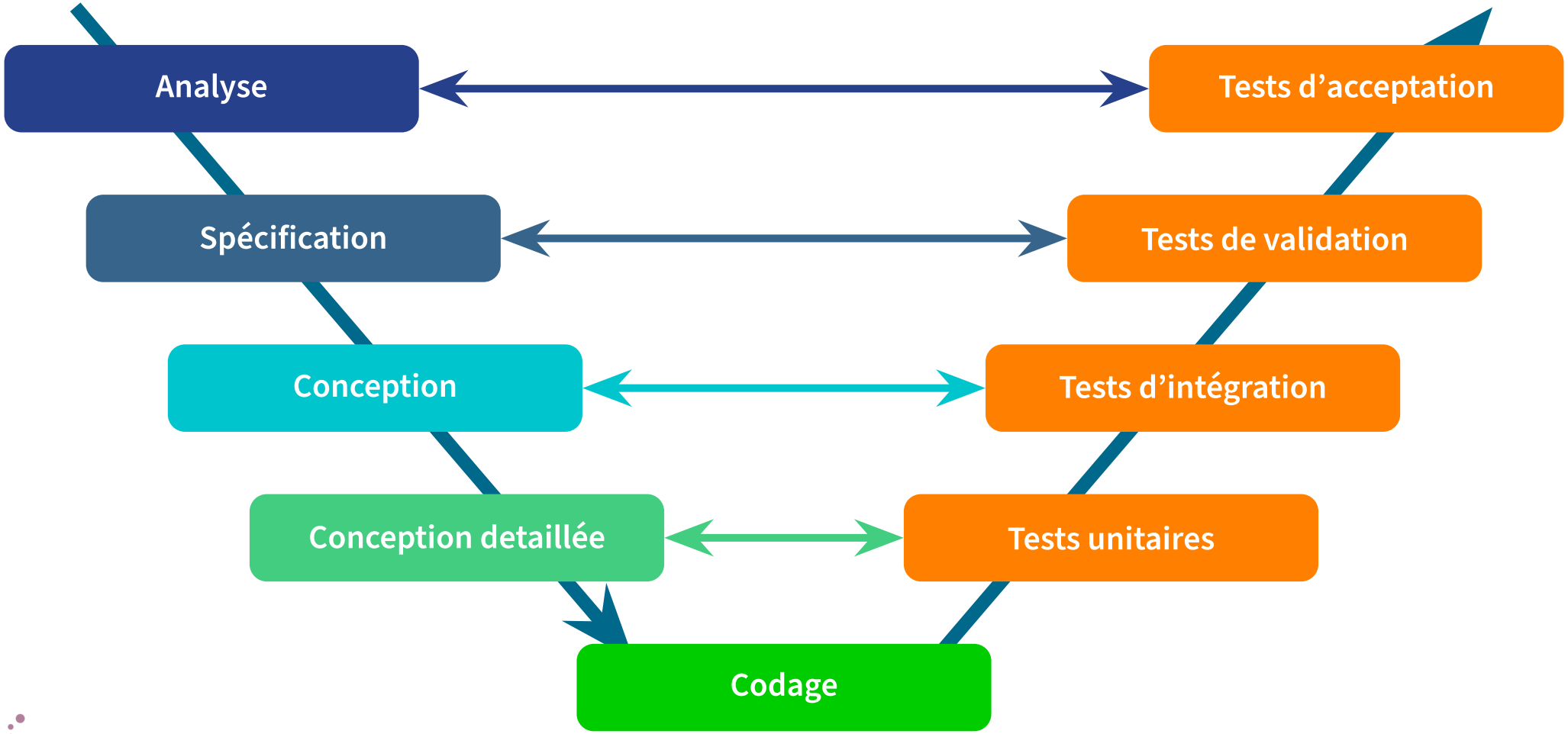
# CONCEPTION ORIENTÉE OBJET - COO

## LES TESTS UNITAIRES EN JAVA

2A - Bachelor Universitaire de Technologie
IUT d'Orsay - Université Paris-Saclay - 2025/2026

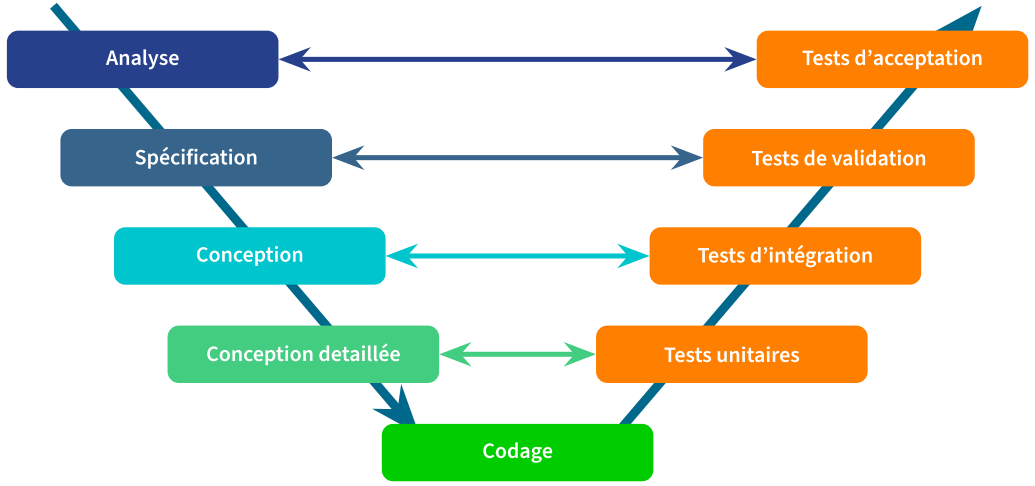
**Idir AIT SADOUNE**
idir.ait-sadoune@universite-paris-saclay.fr

# PROCESSUS DE DÉVELOPPEMENT EN V



# LES ACTIVITÉS DE TESTS

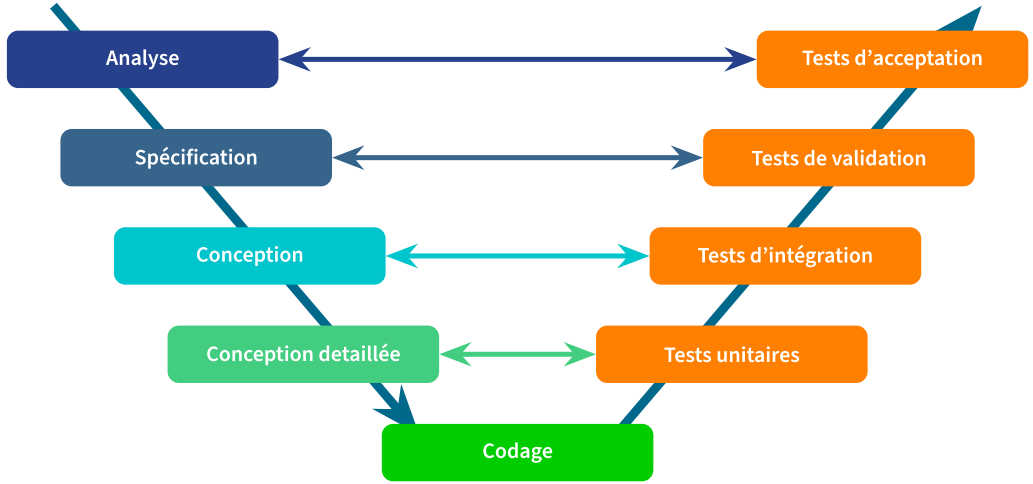
1. **Les tests unitaires** $\longrightarrow$ vérifier séparément le fonctionnement des modules d'un logiciel.



# LES ACTIVITÉS DE TESTS

1. **Les tests unitaires** $\longrightarrow$ vérifier séparément le fonctionnement des modules d'un logiciel.

2. **Les tests d'intégration** $\longrightarrow$ regrouper l'ensemble des modules et valider le fonctionnement global du logiciel.

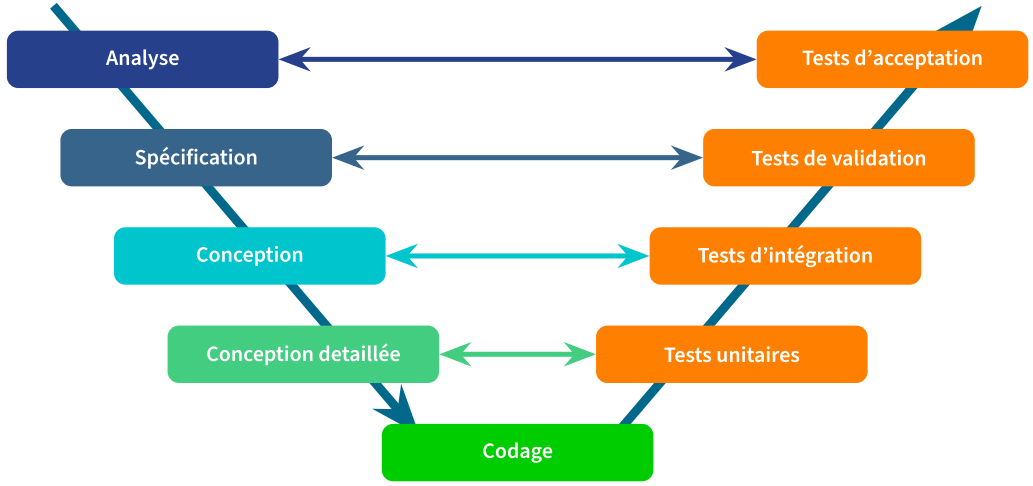


# LES ACTIVITÉS DE TESTS

1. **Les tests unitaires** $\longrightarrow$ vérifier séparément le fonctionnement des modules d'un logiciel.

2. **Les tests d'intégration** $\longrightarrow$ regrouper l'ensemble des modules et valider le fonctionnement global du logiciel.

3. **Les tests de validation** $\longrightarrow$ valider la conformité du logiciel par rapport aux spécifications.



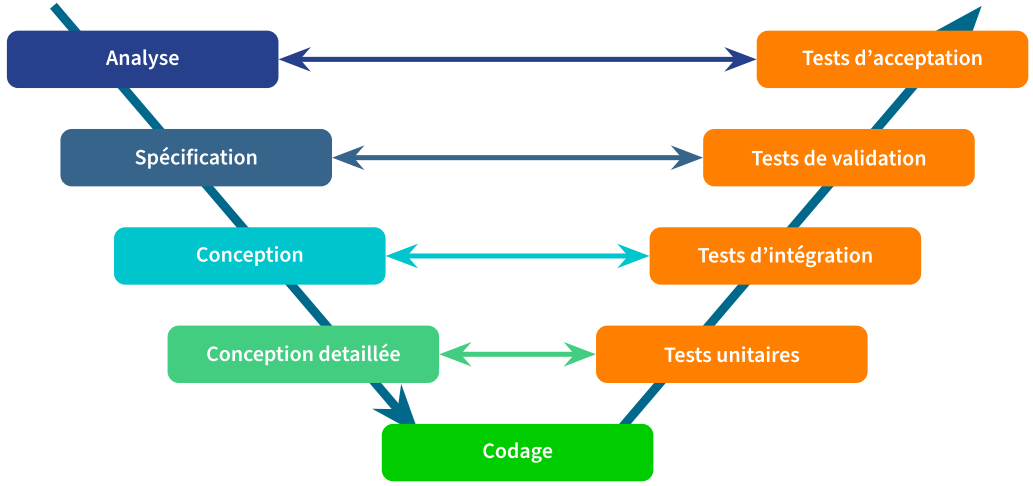
# LES ACTIVITÉS DE TESTS

1. **Les tests unitaires** $\longrightarrow$ vérifier séparément le fonctionnement des modules d'un logiciel.

2. **Les tests d'intégration** $\longrightarrow$ regrouper l'ensemble des modules et valider le fonctionnement global du logiciel.

3. **Les tests de validation** $\longrightarrow$ valider la conformité du logiciel par rapport aux spécifications.

4. **Les tests d'acceptation** $\longrightarrow$ vérifier avec le client que la solution proposée est conforme à ses exigences.



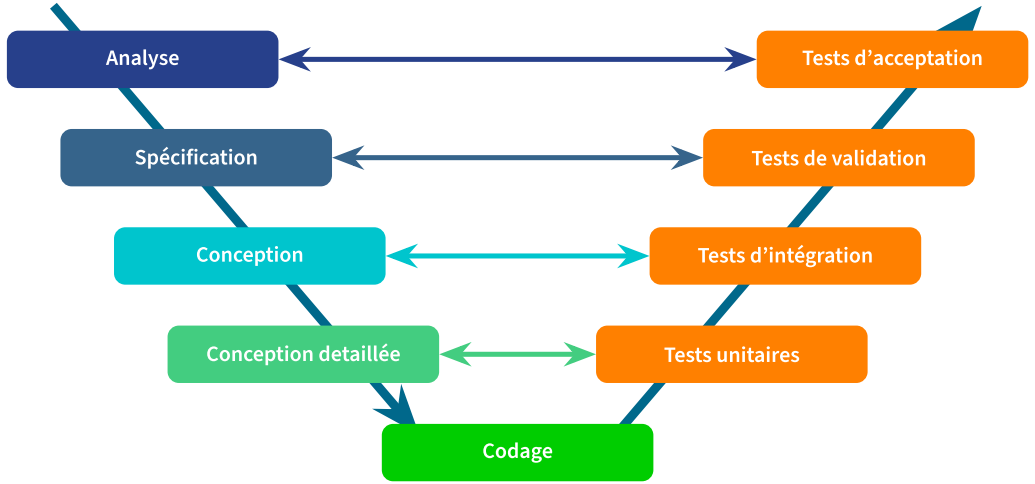
# DANS CE COURS

1. **Les tests unitaires** $\longrightarrow$ vérifier séparément le fonctionnement des modules d'un logiciel.

2. **Les tests d'intégration** $\longrightarrow$ regrouper l'ensemble des modules et valider le fonctionnement global du logiciel.

3. **Les tests de validation** $\longrightarrow$ valider la conformité du logiciel par rapport aux spécifications.

4. **Les tests d'acceptation** $\longrightarrow$ vérifier avec le client que la solution proposée est conforme à ses exigences.



# LA PHILOSOPHIE DU TEST

- Pas de différence entre test et débogage.
  - ➠ déboguer avec des jeux d'entrées

# LA PHILOSOPHIE DU TEST

- Pas de différence entre test et débogage.
  - ➠ déboguer avec des jeux d'entrées
- Le test sert à montrer que le logiciel fonctionne.
  - ➠ exécuter une suite de tests sans défaillance

# LA PHILOSOPHIE DU TEST

- Pas de différence entre test et débogage.
  - ➠ déboguer avec des jeux d'entrées
- Le test sert à montrer que le logiciel fonctionne.
  - ➠ exécuter une suite de tests sans défaillance
- Le test montre la présence d'erreurs mais pas leur absence.
  - ➠ permettre de diminuer le risque.

# LES TESTS UNITAIRES

- **Un test unitaire** est **un script** permettant de **valider** le fonctionnement d'une partie d'un module (une méthode, une class, une interface, ...).

# LES TESTS UNITAIRES

- **Un test unitaire** est **un script** permettant de **valider** le fonctionnement d'une partie d'un module (une méthode, une class, une interface, ...).
- En **Java**, le framework (JUnit), offre des fonctionnalités permettant le développement des tests unitaires.

# LES ANNOTATIONS

**JUnit** utilise un système d'annotations définissant le comportement d'une méthode dans **une classe de test** :

# LES ANNOTATIONS

**JUnit** utilise un système d'annotations définissant le comportement d'une méthode dans **une classe de test** :

- `@Test` : spécifie que la méthode annotée est un test.

# LES ANNOTATIONS

**JUnit** utilise un système d'annotations définissant le comportement d'une méthode dans **une classe de test** :

- `@Test` : spécifie que la méthode annotée est un test.
- `@Before` ou `@BeforeEach` (JUnit5) : spécifie que la méthode annotée sera exécutée avant chaque méthode de test.

# LES ANNOTATIONS

**JUnit** utilise un système d'annotations définissant le comportement d'une méthode dans **une classe de test** :

- `@Test` : spécifie que la méthode annotée est un test.
- `@Before` ou `@BeforeEach` (JUnit5) : spécifie que la méthode annotée sera exécutée avant chaque méthode de test.
- `@BeforeClass` ou `@BeforeAll` (JUnit5): spécifie que la méthode annotée sera exécutée une fois avant toutes les méthodes de test.

# LES ANNOTATIONS

**JUnit** utilise un système d'annotations définissant le comportement d'une méthode dans **une classe de test** :

- `@Test` : spécifie que la méthode annotée est un test.
- `@Before` ou `@BeforeEach` (JUnit5) : spécifie que la méthode annotée sera exécutée avant chaque méthode de test.
- `@BeforeClass` ou `@BeforeAll` (JUnit5): spécifie que la méthode annotée sera exécutée une fois avant toutes les méthodes de test.
- `@After` ou `@AfterEach` (JUnit5) : spécifie que la méthode annotée sera exécutée après chaque méthode de test.

# LES ANNOTATIONS

**JUnit** utilise un système d'annotations définissant le comportement d'une méthode dans **une classe de test** :

- `@Test` : spécifie que la méthode annotée est un test.
- `@Before` ou `@BeforeEach` (JUnit5) : spécifie que la méthode annotée sera exécutée avant chaque méthode de test.
- `@BeforeClass` ou `@BeforeAll` (JUnit5): spécifie que la méthode annotée sera exécutée une fois avant toutes les méthodes de test.
- `@After` ou `@AfterEach` (JUnit5) : spécifie que la méthode annotée sera exécutée après chaque méthode de test.
- `@AfterClass` ou `@AfterAll` (JUnit5): spécifie que la méthode annotée sera exécutée une fois après toutes les méthodes de test.

# LA CLASSE ASSERTIONS

Lien vers la documentation de la classe Assertions

# LA CLASSE ASSERTIONS

- `void assertTrue(boolean condition)`
  vérifie qu'une condition est vraie.

# LA CLASSE ASSERTIONS

- `void assertTrue(boolean condition)`
  vérifie qu'une condition est vraie.
- `void assertFalse(boolean condition)`
  vérifie qu'une condition est fausse.

# LA CLASSE ASSERTIONS

- `void assertTrue(boolean condition)`
  vérifie qu'une condition est vraie.
- `void assertFalse(boolean condition)`
  vérifie qu'une condition est fausse.
- `void assertNotNull(Object object)`
  vérifie qu'un objet n'est pas null.

# LA CLASSE ASSERTIONS

- `void assertTrue(boolean condition)`
  vérifie qu'une condition est vraie.
- `void assertFalse(boolean condition)`
  vérifie qu'une condition est fausse.
- `void assertNotNull(Object object)`
  vérifie qu'un objet n'est pas null.
- `void assertNull(Object object)`
  vérifie qu'un objet est null.

# LA CLASSE ASSERTIONS

- `void assertTrue(boolean condition)`
  vérifie qu'une condition est vraie.
- `void assertFalse(boolean condition)`
  vérifie qu'une condition est fausse.
- `void assertNotNull(Object object)`
  vérifie qu'un objet n'est pas null.
- `void assertNull(Object object)`
  vérifie qu'un objet est null.
- `void assertEquals(Object expected, Object actual)`
  vérifie l'égalité entre deux objets ou deux types primitifs.

# LA CLASSE ASSERTIONS

- ...
- `void assertSame(Object expected, Object actual)`
  vérifie l'égalité entre deux références d'objets.

# LA CLASSE ASSERTIONS

- ...
- `void assertSame(Object expected, Object actual)`
  vérifie l'égalité entre deux références d'objets.
- `void assertNotSame(Object expected, Object actual)`
  vérifie que deux références d'objets sont différentes.

# LA CLASSE ASSERTIONS

- ...
- `void assertSame(Object expected, Object actual)`
  vérifie l'égalité entre deux références d'objets.
- `void assertNotSame(Object expected, Object actual)`
  vérifie que deux références d'objets sont différentes.
- `void assertArrayEquals(expectedArray, resultArray)`
  vérifie que deux tableaux sont égaux.

# LA CLASSE ASSERTIONS

- ...
- `void assertSame(Object expected, Object actual)`
  vérifie l'égalité entre deux références d'objets.
- `void assertNotSame(Object expected, Object actual)`
  vérifie que deux références d'objets sont différentes.
- `void assertArrayEquals(expectedArray, resultArray)`
  vérifie que deux tableaux sont égaux.
- `void fail(String)`
  force un test à l'échec.

# CODE JAVA À TESTER

```java
public class Contact {
    private String firstName;
    private String lastName;
    private String phoneNumber;

    public Contact(String firstName, String lastName, String phoneNumber) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.phoneNumber = phoneNumber;
    }

    public String getFirstName() {
        return firstName;
    }

    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }

    public String getLastName() {
        return lastName;
    }
```

université PARIS-SACLAY
IUT D'ORSAY

# CODE JAVA À TESTER

```java
public class Contact {
    private String firstName;
    private String lastName;
    private String phoneNumber;

    public Contact(String firstName, String lastName, String phoneNumber) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.phoneNumber = phoneNumber;
    }

    public String getFirstName() {
        return firstName;
    }

    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }

    public String getLastName() {
        return lastName;
    }
```

université PARIS-SACLAY
IUT D'ORSAY

# CODE JAVA À TESTER

```java
    private String phoneNumber;

    public Contact(String firstName, String lastName, String phoneNumber) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.phoneNumber = phoneNumber;
    }

    public String getFirstName() {
        return firstName;
    }

    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }

    public String getLastName() {
        return lastName;
    }

    public void setLastName(String lastName) {
        this.lastName = lastName;
    }
```

université PARIS-SACLAY
IUT D'ORSAY

# CODE JAVA À TESTER

```java
    public String getFirstName() {
        return firstName;
    }

    public void setFirstName(String firstName) {
        this.firstName = firstName;
    }

    public String getLastName() {
        return lastName;
    }

    public void setLastName(String lastName) {
        this.lastName = lastName;
    }

    public String getPhoneNumber() {
        return phoneNumber;
    }

    public void setPhoneNumber(String phoneNumber) {
        this.phoneNumber = phoneNumber;
    }
```

université PARIS-SACLAY
IUT D'ORSAY

# CODE JAVA À TESTER

```java
    public String getLastName() {
        return lastName;
    }

    public void setLastName(String lastName) {
        this.lastName = lastName;
    }

    public String getPhoneNumber() {
        return phoneNumber;
    }

    public void setPhoneNumber(String phoneNumber) {
        this.phoneNumber = phoneNumber;
    }

    public void validateFirstName() {
        if (this.firstName.isBlank())
            throw new RuntimeException("First Name Cannot be null or empty");
    }

    public void validateLastName() {
        if (this.lastName.isBlank())
```

université PARIS-SACLAY
IUT D'ORSAY

# CODE JAVA À TESTER

```
27
28      public String getPhoneNumber() {
29          return phoneNumber;
30      }
31
32      public void setPhoneNumber(String phoneNumber) {
33          this.phoneNumber = phoneNumber;
34      }
35
36      public void validateFirstName() {
37          if (this.firstName.isBlank())
38              throw new RuntimeException("First Name Cannot be null or empty");
39      }
40
41      public void validateLastName() {
42          if (this.lastName.isBlank())
43              throw new RuntimeException("Last Name Cannot be null or empty");
44      }
45
46      public void validatePhoneNumber() {
47          if (this.phoneNumber.isBlank()){
48              throw new RuntimeException("Phone Name Cannot be null or empty");
```

université PARIS-SACLAY
IUT D'ORSAY

# CODE JAVA À TESTER

```java
    public void setPhoneNumber(String phoneNumber) {
        this.phoneNumber = phoneNumber;
    }

    public void validateFirstName() {
        if (this.firstName.isBlank())
            throw new RuntimeException("First Name Cannot be null or empty");
    }

    public void validateLastName() {
        if (this.lastName.isBlank())
            throw new RuntimeException("Last Name Cannot be null or empty");
    }

    public void validatePhoneNumber() {
        if (this.phoneNumber.isBlank()){
            throw new RuntimeException("Phone Name Cannot be null or empty");
        }
        if (this.phoneNumber.length() != 10) {
            throw new RuntimeException("Phone Number Should be 10 Digits Long");
        }
        if (!this.phoneNumber.matches("\d+")) {
```

université PARIS-SACLAY
IUT D'ORSAY

# CODE JAVA À TESTER

```java
    }

    public void validateLastName() {
        if (this.lastName.isBlank())
            throw new RuntimeException("Last Name Cannot be null or empty");
    }

    public void validatePhoneNumber() {
        if (this.phoneNumber.isBlank()){
            throw new RuntimeException("Phone Name Cannot be null or empty");
        }
        if (this.phoneNumber.length() != 10) {
            throw new RuntimeException("Phone Number Should be 10 Digits Long");
        }
        if (!this.phoneNumber.matches("\d+")) {
            throw new RuntimeException("Phone Number Contain only digits");
        }
        if (!this.phoneNumber.startsWith("0")) {
            throw new RuntimeException("Phone Number Should Start with 0");
        }
    }
}
```

université PARIS-SACLAY
IUT D'ORSAY

# CODE JAVA À TESTER

```java
import java.util.Collection;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class ContactManager {

    Map<String, Contact> contactList = new ConcurrentHashMap<String, Contact>();

    public void addContact(String firstName, String lastName, String phoneNumber) {
        Contact contact = new Contact(firstName, lastName, phoneNumber);
        this.validateContact(contact);
        this.checkIfContactAlreadyExist(contact);
        this.contactList.put(this.generateKey(contact), contact);
    }

    public Collection<Contact> getAllContacts() {
        return this.contactList.values();
    }

    private void checkIfContactAlreadyExist(Contact contact) {
        if (this.contactList.containsKey(this.generateKey(contact)))
            throw new RuntimeException("Contact Already Exists");
    }
```

# CODE JAVA À TESTER

```java
import java.util.Collection;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class ContactManager {

    Map<String, Contact> contactList = new ConcurrentHashMap<String, Contact>();

    public void addContact(String firstName, String lastName, String phoneNumber) {
        Contact contact = new Contact(firstName, lastName, phoneNumber);
        this.validateContact(contact);
        this.checkIfContactAlreadyExist(contact);
        this.contactList.put(this.generateKey(contact), contact);
    }

    public Collection<Contact> getAllContacts() {
        return this.contactList.values();
    }

    private void checkIfContactAlreadyExist(Contact contact) {
        if (this.contactList.containsKey(this.generateKey(contact)))
            throw new RuntimeException("Contact Already Exists");
    }
```

université PARIS-SACLAY
IUT D'ORSAY

# CODE JAVA À TESTER

```java
import java.util.Collection;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

public class ContactManager {

    Map<String, Contact> contactList = new ConcurrentHashMap<String, Contact>();

    public void addContact(String firstName, String lastName, String phoneNumber) {
        Contact contact = new Contact(firstName, lastName, phoneNumber);
        this.validateContact(contact);
        this.checkIfContactAlreadyExist(contact);
        this.contactList.put(this.generateKey(contact), contact);
    }

    public Collection<Contact> getAllContacts() {
        return this.contactList.values();
    }

    private void checkIfContactAlreadyExist(Contact contact) {
        if (this.contactList.containsKey(this.generateKey(contact)))
            throw new RuntimeException("Contact Already Exists");
    }
```

# CODE JAVA À TESTER

```java

    Map<String, Contact> contactList = new ConcurrentHashMap<String, Contact>();

    public void addContact(String firstName, String lastName, String phoneNumber) {
        Contact contact = new Contact(firstName, lastName, phoneNumber);
        this.validateContact(contact);
        this.checkIfContactAlreadyExist(contact);
        this.contactList.put(this.generateKey(contact), contact);
    }

    public Collection<Contact> getAllContacts() {
        return this.contactList.values();
    }

    private void checkIfContactAlreadyExist(Contact contact) {
        if (this.contactList.containsKey(this.generateKey(contact)))
            throw new RuntimeException("Contact Already Exists");
    }

    private void validateContact(Contact contact) {
        contact.validateFirstName();
        contact.validateLastName();
        contact.validatePhoneNumber();
```

université PARIS-SACLAY
IUT D'ORSAY

# CODE JAVA À TESTER

```java
        Contact contact = new Contact(firstName, lastName, phoneNumber);
        this.validateContact(contact);
        this.checkIfContactAlreadyExist(contact);
        this.contactList.put(this.generateKey(contact), contact);
    }

    public Collection<Contact> getAllContacts() {
        return this.contactList.values();
    }

    private void checkIfContactAlreadyExist(Contact contact) {
        if (this.contactList.containsKey(this.generateKey(contact)))
            throw new RuntimeException("Contact Already Exists");
    }

    private void validateContact(Contact contact) {
        contact.validateFirstName();
        contact.validateLastName();
        contact.validatePhoneNumber();
    }

    private String generateKey(Contact contact) {
        return String.format("%s-%s", contact.getFirstName(), contact.getLastName());
    }
```

# CODE JAVA À TESTER

```java
        this.checkIfContactAlreadyExist(contact);
        this.contactList.put(this.generateKey(contact), contact);
    }

    public Collection<Contact> getAllContacts() {
        return this.contactList.values();
    }

    private void checkIfContactAlreadyExist(Contact contact) {
        if (this.contactList.containsKey(this.generateKey(contact)))
            throw new RuntimeException("Contact Already Exists");
    }

    private void validateContact(Contact contact) {
        contact.validateFirstName();
        contact.validateLastName();
        contact.validatePhoneNumber();
    }

    private String generateKey(Contact contact) {
        return String.format("%s-%s", contact.getFirstName(), contact.getLastName());
    }
}
```

université PARIS-SACLAY
IUT D'ORSAY

# CODE JAVA À TESTER

```java
        this.checkIfContactAlreadyExist(contact);
        this.contactList.put(this.generateKey(contact), contact);
    }

    public Collection<Contact> getAllContacts() {
        return this.contactList.values();
    }

    private void checkIfContactAlreadyExist(Contact contact) {
        if (this.contactList.containsKey(this.generateKey(contact)))
            throw new RuntimeException("Contact Already Exists");
    }

    private void validateContact(Contact contact) {
        contact.validateFirstName();
        contact.validateLastName();
        contact.validatePhoneNumber();
    }

    private String generateKey(Contact contact) {
        return String.format("%s-%s", contact.getFirstName(), contact.getLastName());
    }
}
```

université PARIS-SACLAY
IUT D'ORSAY

# PREMIER TEST JUNIT5

```java
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

public class ContactManagerTest {

  @Test
  public void shouldCreateContact() {
      ContactManager contactManager = new ContactManager();
      contactManager.addContact("John", "Doe", "0123456789");

      assertTrue(! contactManager.getAllContacts().isEmpty());
      assertFalse(contactManager.getAllContacts().isEmpty());
      assertEquals(1, contactManager.getAllContacts().size());
  }
}
```

# CYCLE DE VIE D'UN TEST JUNIT

```java
1  import org.junit.jupiter.api.*;
2  import org.junit.jupiter.api.condition.EnabledOnOs;
3  import org.junit.jupiter.api.condition.OS;
4  import org.junit.jupiter.params.ParameterizedTest;
5  import org.junit.jupiter.params.provider.CsvFileSource;
6  import org.junit.jupiter.params.provider.CsvSource;
7  import org.junit.jupiter.params.provider.MethodSource;
8  import org.junit.jupiter.params.provider.ValueSource;
9
10 import java.util.Arrays;
11 import java.util.List;
12
13 import static org.junit.jupiter.api.Assertions.assertEquals;
14 import static org.junit.jupiter.api.Assertions.assertFalse;
15
16 public class ContactManagerTest {
17
18     private ContactManager contactManager;
19
20     @BeforeAll
21     public static void setupAll() {
22         System.out.println("Should Print Before All Tests");
```

université PARIS-SACLAY
IUT D'ORSAY

# CYCLE DE VIE D'UN TEST JUNIT

```java
import org.junit.jupiter.params.provider.MethodSource;
import org.junit.jupiter.params.provider.ValueSource;

import java.util.Arrays;
import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertFalse;

public class ContactManagerTest {

    private ContactManager contactManager;

    @BeforeAll
    public static void setupAll() {
        System.out.println("Should Print Before All Tests");
    }

    @BeforeEach
    public void setup() {
        System.out.println("Instantiating Contact Manager");
        contactManager = new ContactManager();
    }
```

université PARIS-SACLAY
IUT D'ORSAY

# CYCLE DE VIE D'UN TEST JUNIT

```java
14 import static org.junit.jupiter.api.Assertions.assertFalse;
15
16 public class ContactManagerTest {
17
18     private ContactManager contactManager;
19
20     @BeforeAll
21     public static void setupAll() {
22         System.out.println("Should Print Before All Tests");
23     }
24
25     @BeforeEach
26     public void setup() {
27         System.out.println("Instantiating Contact Manager");
28         contactManager = new ContactManager();
29     }
30
31     @Test
32     @DisplayName("Should Create Contact")
33     public void shouldCreateContact() {
34         contactManager.addContact("John", "Doe", "0123456789");
35         assertFalse(contactManager.getAllContacts().isEmpty());
```

# CYCLE DE VIE D'UN TEST JUNIT

```
23      }
24
25      @BeforeEach
26      public void setup() {
27          System.out.println("Instantiating Contact Manager");
28          contactManager = new ContactManager();
29      }
30
31      @Test
32      @DisplayName("Should Create Contact")
33      public void shouldCreateContact() {
34          contactManager.addContact("John", "Doe", "0123456789");
35          assertFalse(contactManager.getAllContacts().isEmpty());
36          assertEquals(1, contactManager.getAllContacts().size());
37      }
38
39      @Test
40      @DisplayName("Should Not Create Contact When First Name is Null")
41      public void shouldThrowRuntimeExceptionWhenFirstNameIsNull() {
42          Assertions.assertThrows(RuntimeException.class, () -> {
43              contactManager.addContact(null, "Doe", "0123456789");
44          });
45      }
```

université PARIS-SACLAY
IUT D'ORSAY

# CYCLE DE VIE D'UN TEST JUNIT

```
141     private static List<String> phoneNumberList() {
142         return Arrays.asList("0123456789", "0123456798", "0123456897");
143     }
144
145     @Test
146     @DisplayName("Test Should Be Disabled")
147     @Disabled
148     public void shouldBeDisabled() {
149         throw new RuntimeException("Test Should Not be executed");
150     }
151
152
153     @AfterEach
154     public void tearDown() {
155         System.out.println("Should Execute After Each Test");
156     }
157
158     @AfterAll
159     public static void tearDownAll() {
160         System.out.println("Should be executed at the end of the Test");
161     }
162 }
```

université PARIS-SACLAY
IUT D'ORSAY

# RÉPÉTER DES TESTS AVEC JUNIT

```
83      public void shouldTestPhoneNumberFormat() {
84          contactManager.addContact("John", "Doe", "0123456789");
85          assertFalse(contactManager.getAllContacts().isEmpty());
86          assertEquals(1, contactManager.getAllContacts().size());
87      }
88
89
90      @Nested
91      class RepeatedTests {
92      @DisplayName("Repeat Contact Creation Test 5 Times")
93      @RepeatedTest(value = 5,
94      name = "Repeating Contact Creation Test {currentRepetition} of {totalRepetitions}")
95          public void shouldTestContactCreationRepeatedly() {
96              contactManager.addContact("John", "Doe", "0123456789");
97              assertFalse(contactManager.getAllContacts().isEmpty());
98              assertEquals(currentRepetition, contactManager.getAllContacts().size());
99          }
100     }
101
102     @Nested
103     class ParameterizedTests {
104         @DisplayName("Phone Number should match the required Format")
105         @ParameterizedTest
106         @ValueSource(strings = {"0123456789", "0123456798", "0123456897"})
107         public void shouldTestPhoneNumberFormatUsingValueSource(String phoneNumber) {
```

université PARIS-SACLAY
IUT D'ORSAY

# LES TESTS PARAMÉTRÉS

```
95          public void shouldTestContactCreationRepeatedly() {
96              contactManager.addContact("John", "Doe", "0123456789");
97              assertFalse(contactManager.getAllContacts().isEmpty());
98              assertEquals(1, contactManager.getAllContacts().size());
99          }
100     }
101
102     @Nested
103     class ParameterizedTests {
104         @DisplayName("Phone Number should match the required Format")
105         @ParameterizedTest
106         @ValueSource(strings = {"0123456789", "0123456798", "0123456897"})
107         public void shouldTestUsingValueSource(String phoneNumber) {
108             contactManager.addContact("John", "Doe", phoneNumber);
109             assertFalse(contactManager.getAllContacts().isEmpty());
110             assertNotEquals(0, contactManager.getAllContacts().size());
111         }
112
113         @DisplayName("CSV Source Case - Phone Number should match the required Format")
114         @ParameterizedTest
115         @CsvSource({"John, Doe, 0123456789", "Bill, Jack, 0123456798", "Maria, Jack, 01234
116         public void shouldTestUsingCSVSource(String fn, String ln, String pn) {
117             contactManager.addContact(fn, ln, pn);
118             assertFalse(contactManager.getAllContacts().isEmpty());
```

université PARIS-SACLAY
IUT D'ORSAY

# LES TESTS PARAMÉTRÉS

```
105         @ParameterizedTest
106         @ValueSource(strings = {"0123456789", "0123456798", "0123456897"})
107         public void shouldTestUsingValueSource(String phoneNumber) {
108             contactManager.addContact("John", "Doe", phoneNumber);
109             assertFalse(contactManager.getAllContacts().isEmpty());
110             assertNotEquals(0, contactManager.getAllContacts().size());
111         }
112
113         @DisplayName("CSV Source Case - Phone Number should match the required Format")
114         @ParameterizedTest
115         @CsvSource({"John, Doe, 0123456789", "Bill, Jack, 0123456798", "Maria, Jack, 01234
116         public void shouldTestUsingCSVSource(String fn, String ln, String pn) {
117             contactManager.addContact(fn, ln, pn);
118             assertFalse(contactManager.getAllContacts().isEmpty());
119             assertNotEquals(0, contactManager.getAllContacts().size());
120         }
121
122         @DisplayName("CSV File Source Case - Phone Number should match the required Format
123         @ParameterizedTest
124         @CsvFileSource(resources = "/data.csv")
125         public void shouldTestUsingCSVFileSource(String fn, String ln, String pn) {
126             contactManager.addContact(fn, ln, pn);
127             assertFalse(contactManager.getAllContacts().isEmpty());
128             assertNotEquals(0, contactManager.getAllContacts().size());
```

université PARIS-SACLAY
IUT D'ORSAY

# LES TESTS PARAMÉTRÉS

```
114         @ParameterizedTest
115         @CsvSource({"John, Doe, 0123456789", "Bill, Jack, 0123456798", "Maria, Jack, 01234
116         public void shouldTestUsingCSVSource(String fn, String ln, String pn) {
117             contactManager.addContact(fn, ln, pn);
118             assertFalse(contactManager.getAllContacts().isEmpty());
119             assertNotEquals(0, contactManager.getAllContacts().size());
120         }
121
122         @DisplayName("CSV File Source Case - Phone Number should match the required Format
123         @ParameterizedTest
124         @CsvFileSource(resources = "/data.csv")
125         public void shouldTestUsingCSVFileSource(String fn, String ln, String pn) {
126             contactManager.addContact(fn, ln, pn);
127             assertFalse(contactManager.getAllContacts().isEmpty());
128             assertNotEquals(0, contactManager.getAllContacts().size());
129         }
130     }
131
132     @DisplayName("Method Source Case - Phone Number should match the required Format")
133     @ParameterizedTest
134     @MethodSource("phoneNumberList")
135     public void shouldTestPhoneNumberFormatUsingMethodSource(String phoneNumber) {
136         contactManager.addContact("John", "Doe", phoneNumber);
137         assertFalse(contactManager.getAllContacts().isEmpty());
```

université PARIS-SACLAY
IUT D'ORSAY

# LES TESTS PARAMÉTRÉS

```
126             contactManager.addContact(fn, ln, pn);
127             assertFalse(contactManager.getAllContacts().isEmpty());
128             assertNotEquals(0, contactManager.getAllContacts().size());
129         }
130     }
131
132     @DisplayName("Method Source Case - Phone Number should match the required Format")
133     @ParameterizedTest
134     @MethodSource("phoneNumberList")
135     public void shouldTestPhoneNumberFormatUsingMethodSource(String phoneNumber) {
136         contactManager.addContact("John", "Doe", phoneNumber);
137         assertFalse(contactManager.getAllContacts().isEmpty());
138         assertEquals(1, contactManager.getAllContacts().size());
139     }
140
141     private static List<String> phoneNumberList() {
142         return Arrays.asList("0123456789", "0123456798", "0123456897");
143     }
144
145     @Test
146     @DisplayName("Test Should Be Disabled")
147     @Disabled
148     public void shouldBeDisabled() {
149         throw new RuntimeException("Test Should Not be executed");
```

université PARIS-SACLAY
IUT D'ORSAY

# LES TESTS IGNORÉS

```
137         assertFalse(contactManager.getAllContacts().isEmpty());
138         assertEquals(1, contactManager.getAllContacts().size());
139     }
140
141     private static List<String> phoneNumberList() {
142         return Arrays.asList("0123456789", "0123456798", "0123456897");
143     }
144
145     @Test
146     @DisplayName("Test Should Be Disabled")
147     @Disabled
148     public void shouldBeDisabled() {
149         throw new RuntimeException("Test Should Not be executed");
150     }
151
152
153     @AfterEach
154     public void tearDown() {
155         System.out.println("Should Execute After Each Test");
156     }
157
158     @AfterAll
```

université PARIS-SACLAY
IUT D'ORSAY

# TESTS DES EXCEPTIONS AVEC JUNIT

Utilisation de la méthode `assertThrows`

```java
import org.junit.jupiter.api.*;
import static org.junit.jupiter.api.Assertions.*;

public class ContactManagerTest {

@Test
public void shouldThrowRuntimeExceptionWhenFirstNameIsNull() {
    ContactManager contactManager = new ContactManager();

    Assertions.assertThrows(RuntimeException.class, () -> {
        contactManager.addContact(null, "Doe", "0123456789");
    });
}
}
```

université PARIS-SACLAY
IUT D'ORSAY

# MERCI

Version PDF des slides

Retour à l'accueil - Retour au plan